

RAM

.. stands for Retrieval Augmented Generation.

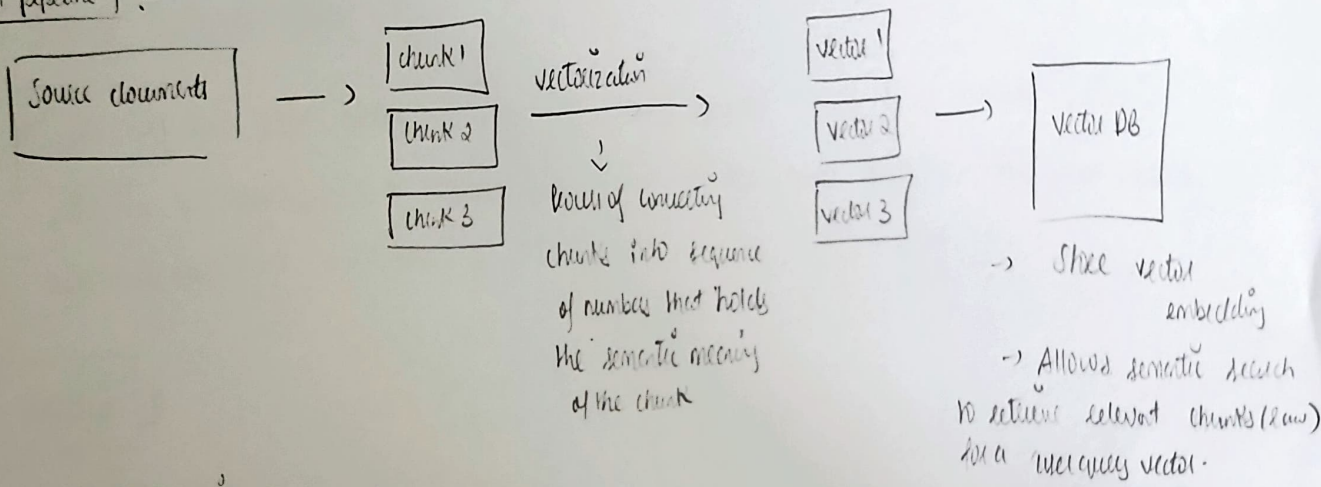
-> Provides access to exclusive data sent as context to the LLMs using which it responds to the user's query

-> Economically better than Fine-tuning

↳ Processing of retaining on LLM with unseen data.

-> Smartly using the space in the context window to add documents relevant to the query instead of flooding it with all the source documents.

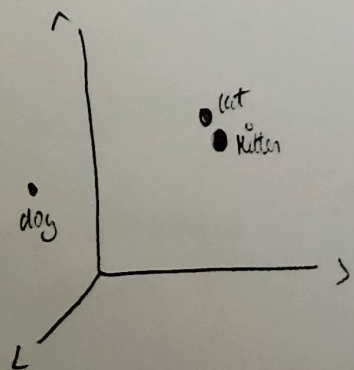
Ingestion pipeline :-



★ Tokens, like how we human see words in a sentence, LLMs see tokens in a sentence.
1 token ~ 1 word.

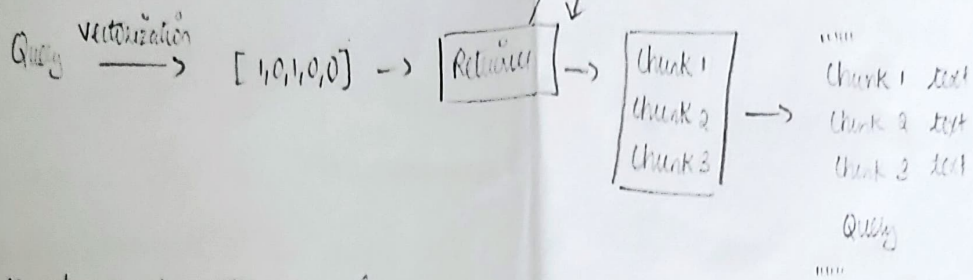
★ When n-dimensional vectors plotted in an n-dimensional space, the terms/tokens/chunks with similar meaning would be closer to each other.

- ★ Cat -> [34, 21, 7, 5]
- ★ Kitten -> [34, 21, 14]
- ★ dog -> [88, 10, 24]



★ When we use modern embedding models to convert chunks or queries to vector embeddings, the embeddings are normalized $\left(\frac{\vec{A}}{\|\vec{A}\|} \right)$, (magnitude is 1)

Retrieval pipeline



★ We convert query to vector embeddings

★ Retrieve all the relevant chunks by performing a semantic search over the vector DB with the converted query.

★ Relevant chunks for a query?

Score $\in [0,1]$

Always use RAG system $\rightarrow$ chunks with the highest similarity score with the query vector calculated using

cosine similarity algorithm are relevant chunks for a query.

★ Must use same embedding model for vectorization of source documents and the user's query.

★ Relevant chunks will be closer to query vector in the multi-dimensional space.

★ Retriever can be configured to give the topk chunks based on the cosine similarity score or filter them by similarity score threshold.

★ Once retrieved, The chunks along with the user's query is passed to the LLM as a prompt

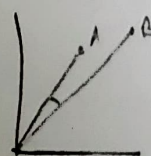
★ dimensions for all the embeddings would be same

cosine similarity :

$$\frac{A \cdot B}{\|A\| \times \|B\|}$$

measures the angle between the A and B vectors

$\downarrow \downarrow$
 $\rightarrow$ embeddings generated with modern embedding models are normalized $\rightarrow \frac{A}{\|A\|}$



$\rightarrow$ So magnitude of a normalized vector will always be 1.

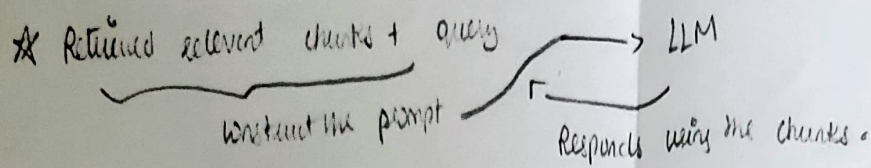
$\therefore$ cosine similarity can be rewritten as $\frac{A \cdot B}{1}$

cosine similarity values ranges from 0 to 1.

... prompt, input prompt, output, answer

Generation :-

(3)



Conversation RAG with chat history :- History-Aware RAG

★ My idea -> For every prompt, also share the past prompts and its response.

★ Actual idea -> Convert the subsequent vague queries to standalone queries with the help of conversation history.

★ We use the chat history containing the user queries and LLM's responses, interact with the LLM and ask it to understand the intent of the user based on the chat history and ask it to construct a standalone question.

Chunking Strategies :-

Problems with character based chunking,

★ Splits mid-sentence

★ Breaks related content apart

★ Context is lost across the chunks.

Leads to poor retrieval quality.

Poor chunks -> Poor response generation

chunk size should not be too small -> loses context awareness
not be too large -> too much noise.

5 chunking strategies

① Character text splitter (Beyond basic chunk size configuration)

↳ split over custom separator (custom pattern) like ".\n\n"

②

ii) Recursive Character Text Splitter :-

4

★ Chunks at natural boundaries (paragraphs, sentences, words), fallbacks if exceeds the chunk limit size.

iii) Document structure splitting (splitting by respecting the document structure)

★ PDFs -> Splits by chapters, pages, sections, headers

★ Markdown -> splits by headers, code blocks, lists, etc...

iv) Semantic splitting :-

★ Use embeddings to detect context shifts between paragraphs or words.

★ Splits when meaning changes, not just by the size.

★ Better results but computationally expensive.

v) Agentic splitting :-

★ splitting boundaries decided using LLMs.

★ Most expensive

Exploring various types of chunking strategy :-

1) Character Text Splitter :-

-> Produces chunk with split fail, greedy search approach. In this chunking strategy, the document is splitted at separator (defaults to: "\n\n", can be custom set) each split is called as a piece and then it merges with the next piece until it does not exceed the chunk limit size.

-> Every chunk produced may not have size = chunk.limit.size.

-> The problem occurs when any piece is greater than the chunk.limit.size, In this case, the piece is not chunked and kept as it is. (as it follows split -> merge, no split because no separator and no merge because the piece size is already greater than the chunk.limit.size).

-> Bad custom separator can split the sentence midway resulting in bad chunk (losing context)

-> This problem is solved by RecursiveCharacterTextSplitter.

ii) Recursive Character Text Splitter:

- Instead of a single separator, this splitter allows us to pass a list of separators.
- default to ["\n\n", "\n", " ", "."] with decreasing priority order.
- It first uses the first separator to produce pieces and if the piece is already greater than the chunk limit size, it splits using the next separator, and so on...

iii) Semantic Chunking:

- Calculates similarity score for every sentence with its next sentence, and merge with the next sentence to form a chunk until we do not see a significant drop in the similarity score (where)
- Uses embedding models to convert sentences to embeddings to calculate similarity scores.
- Economically bad, very expensive.
- We specify the break point percentile, = 70% (40%), we mean, don't split if the similarity is above 70% (40%) of all the similarity scores.

iv) Agentic splitting:

- Using LLM to chunk the document specifying the rules and context in the system prompt.
- Expensive but accurate.

v) Document-Based splitting:

- Using "structured library" for multimodal RAG.
- ↳ PDFs, images, CSV, docs as source documents.

Advanced chunk retrieval strategy :-

- i) Retrieving the top-K relevant chunks based on cosine similarity score with the query vector
- ii) Above technique + ^{minimum} similarity score threshold. (0.3 is the sweet spot)
- iii) Maximum Marginal Relevance (MMR)

NMR: Relevant Material Retrieval

- Avoid chunks with redundant information.
- Provides useful / Rich context to LLMs

LLMs get lost in the middle

6

Step 1 → Find the relevant chunks

Step 2 → Among the relevant chunks, retrieve the diverse chunks

→ Balance between Relevance and Diversity

- ↳ How relevant the chunks are to the query
- ↳ How diverse the chunks are to other chunks.

→ We first fetch K relevant chunks, from it we get $n \times K$ chunks based on a parameter that specifies the balance between the relevance and diversity, called λ (lambda) $\in [0, 1]$

0 → max diversity.

1 → max relevance.

Multi query retrieval to increase the efficiency of our RAG system.

- Generate multiple query variations using LLMs.
- For each variation, perform semantic search and retrieve relevant chunks and store them in separate list
- Now, the chunk in chunks list for each query variation will overlap with other chunks list and may have anomalies.

Reciprocal Rank fusion,

→ Method to retrieve the top chunks from all the chunks list to eventually send as context to LLMs.

→ For every chunk x we calculate its RRF score given by $\sum_{i=1}^n \frac{1}{(b_0 + \text{Position of } x \text{ in } i^{\text{th}} \text{ chunk list})}$

→ Top K chunks with highest RRF scores are the final chunks

RRF ranking

Reducing the positional weightage.

to avoid over penalizing the chunks based on positions.

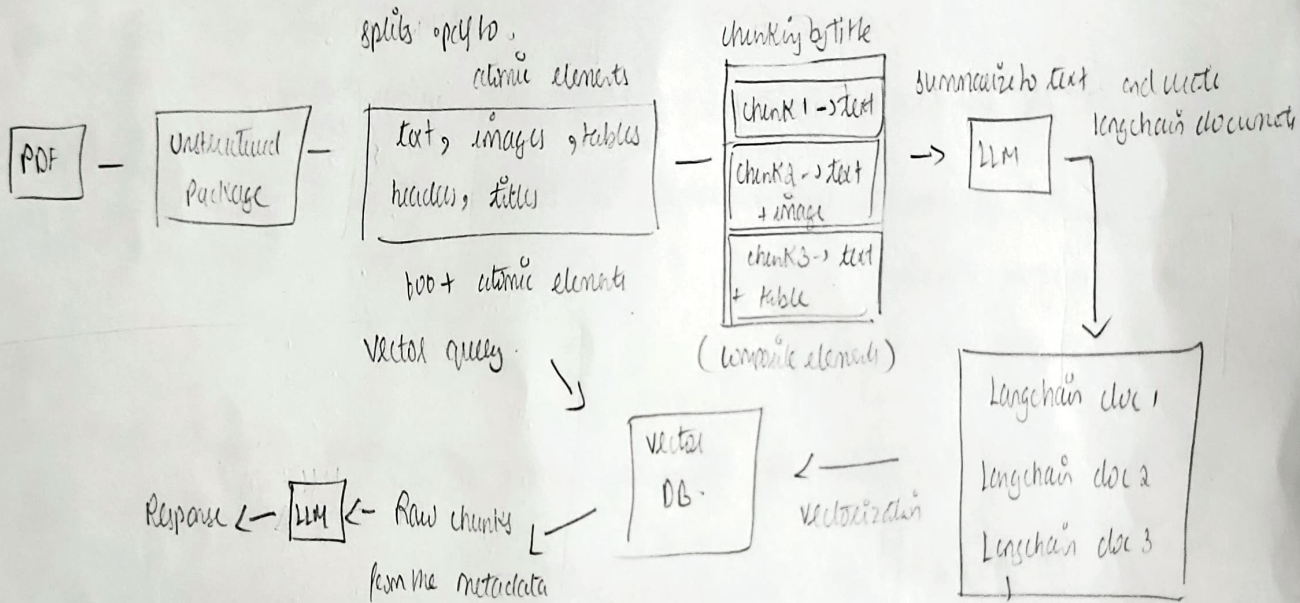
Multi-Modal RAG :-

→ PDFs, DOCX, CSV, website URL.

→ using unstructured library - chunking by title in the case of PDF
using LLMs

→ Hybrid chunks are supposed to be summarized ^{using LLMs} and then vectorized and then stored to retain the semantic meaning information of the image and tables along with the text.
→ Text + media, tables.

→ We cannot convert media like images and tables into vector embeddings.



→ PDF is a highly unstructured data, it consists of images, tables, etc... which cannot be ^{intense of a doc that has a fields metadata, page content} vectorized apart from vectorizable texts. We use unstructured package to split this unstructured document into atomic elements (image, text, headers, title, etc...) and then group these atomic elements under the same title ("introduction", "abstract") to form chunks. This what is called chunking by title. Then we use LLM to generate text summary of each chunk and converted to long documents and then vectorized and then stored. Images are stored in binary format.

Implementation of multimodal RAG:

8

Steps,

- ① Install system wide and project level dependencies.
- ② partition the PDF with unfigit, "hi-tes" parsing method of extraction.
Retain tables as html
Retain images,
store images as base64 format.
- ③ Group atomic elements under same "title" to form composite elements.
- ④ Interact with LLM to generate summary of composite elements.
- ⑤ Form longchain documents, with page_content: LLM response and metadata includes the base64 encoded image and tables as html if any.

Hybrid search:

① Defacto technique Keyword search + Semantic search

② Why hybrid search?

- > Semantic search have blind spots to searches
- > The query may many times explicitly mention the domain terminologies or keywords that are not supposed to be searched based on semantic meaning.

Keyword Search:-

★ BM25 -> standards for Best Matching ds.

★ This algorithm uses 2 key factors:

(TF) - Term frequency -> Frequency of the searched term.

(IDF) - Inverse Document frequency -> How rare the term is within the document.

(3 retrievers)

① Vector retriever

② BM25 retriever

③ Ensemble retriever combines the above retrievers to form a hybrid retriever with adjustable weights.

The weight assigned to each retriever sets the numerator in RRF formula from 1 to the weight and calculates the RRF score for each chunk and based on the ranking, the top chunks are finalized.

④ At scale, the number of chunks retrieved is 50-70 from each retriever if the number of documents are in the order of 10000.

⑤ Multi-query hybrid search:-

